

Home

W1

W2

W3

W4

W5

W6

W7

W05 Learning Activity: Styling Forms

Overview

An HTML form is a user interface that needs styling to be usable and provide a positive user experience. In this activity, you will add CSS to the User Poll form built in the previous activity.

WDD 130 | HTML Form - User Poll

Contact Information

Full Name:

Email:

Questions

What is your favorite internet browser?

Chrome ▾

Do you have a household pet?

☐

List the places you have lived:

Stillwater Florida,
Amsterdam Netherlands,
Tibuktu Mali

Submit Poll

©2024 | HTML Form - User Poll Activity | Xavier Rodriquez

The basic User Poll form from the previous activity with no styling.

Prepare

- Read: [Styling web forms](#) – MDN
- Review: [Best Practices for Form Design](#) – Adobe

Activity Instructions

Using the User Poll form from the previous activity, add CSS to align it with best practices for form design, such as a single-column layout and appropriate padding and spacing in and between form elements.

Here is an example of what you will be producing:

WDD 130 | HTML Form – User Poll

Contact Information

Full Name:

Email:

Questions

What is your favorite internet browser?

Chrome ▾

Do you have a household pet?

☐

List the places you have lived:

Stillwater Florida, Amsterdam Netherlands, Tibuktu Mali

Submit Poll

©2024 | HTML Form – User Poll Activity | Xavier Rodriguez

The same User Poll form with styling added.

Step 1: Setup

1. In the **week05** folder, create a **styles** folder if you have not already.
2. Add a **poll-form.css** file in the **styles** folder.
3. Open the **poll-form.html** file in the **week05** folder.
4. Add the reference link to the CSS file.
5. Start the **Five Server** localhost browser service so that you can watch changes unfold as you add style.

Step 2: Add Style

You are encouraged to design and style the form on your own versus copying the example. The example is provided to give you an idea of what you are aiming for. You will learn more by doing it yourself.

1. Add a font(s) of your choice using the font-family declaration.
2. Style the universal selector `*` rule to set the default **margin** to zero, **padding** to zero, and the **box-sizing** to **border-box**.

▼ Example

```
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
}
```

3. Style the body rule to declare the **font-family** and whatever else you deem necessary

▼ Example

```
body {  
  font-family: Merriweather, Georgia, serif;  
  font-size: 1.2em;  
  color: #333;  
  background: #fff;  
}
```

4. Style the **header** and **h1** as you see fit.

▼ Example

```
header {  
  margin: 0 auto;  
  width: 650px;  
}  
  
h1 {  
  margin: 1rem;  
  font-size: 1.5em;  
  color: navy;
```

```
text-align: center;
}
```

5. Style the **form** with a **width**, a centered **margin**, **padding**, **border**, **border-radius**, **background-color**, and a **display** layout of grid.

▼ Example

```
form {
  width: 650px;
  margin: 0 auto;
  padding: 1rem;
  border: 1px solid rgb(0 0 0 / 0.1);
  border-radius: 0.5rem;
  background-color: #eee;
  display: grid;
}
```

6. Style each **fieldset** to be a **grid** and provide a **gap** and whatever else you deem necessary.

▼ Example

```
fieldset {
  display: grid;
  gap: .5rem;
  margin: 1rem;
  border: 1px solid rgb(0 0 0 / 0.2);
  padding: 1rem;
}
```

7. Style the fieldset **legend**.

▼ Example

```
legend {
  padding: 0.5rem;
  color: navy;
  font-size: 1.3em;
}
```

8. Option: Style all the form elements (**label**, **input**, **button**, **select**, and **textarea**) with a different font family and in whatever other way you see fit that is different than the headings and legends.

▼ Example

```
label, input, button, select, textarea {  
  font-family: Roboto, Arial, sans-serif;  
}
```

9. Style the **label** elements to **text-align** right and have a **padding-right** value.

▼ Example

```
label {  
  text-align: right;  
  padding-right: 1rem;  
}
```

10. Style the **input**, **select**, and **textarea** elements to have **padding** and a larger **font-size**. Note that this is critical in order to provide spacing inside of the form input elements.

▼ Example

```
input, select, textarea {  
  padding: 0.5rem;  
  font-size: 1.1rem;  
}
```

11. Style the checkbox input element to be larger than the default size using **width** and **height**. Change the **vertical-align** to **bottom** to properly align it within its space.

The best practice CSS selector for this is **input[type="checkbox"]**

▼ Example

```
input[type="checkbox"] {  
  width: 22px;
```

```
height: 22px;
vertical-align: bottom;
}
```

12. Style the **button** as you see fit along with the **button:hover** pseudo-class.

▼ Example

```
button {
  margin: 1rem;
  background-color: navy;
  color: #fff;
  border: none;
  border-radius: .5rem;
  padding: .75rem;
  font-size: 1.2rem;
}

button:hover {
  background-color: darkgreen;
}
```

13. Style the **footer**

▼ Example

```
footer {
  margin: 1rem auto 0;
  width: 650px;
  text-align: center;
  font-size: 0.8em;
  color: #777;
  border-top: 1px solid #ccc;
}
```

Step 3: Add HTML Divs

1. Create **div** (container) elements around each form label-input/select pair in your **poll-form.html** document.

There should be five: full name (text input), email (email input), browser

(select), pet (checkbox), and places lived (textarea).

2. Give each of those **div** elements a class attribute named **form-item**. Here is an example on the first item:

```
<div class="form-item">
  <label for="fullname">Full Name:</label>
  <input type="text" id="fullname" name="fullname">
</div>
```

3. Give the fullname and email **div** containers an additional class name of **column2**.

```
<div class="form-item column2">
  <label for="fullname">Full Name:</label>
  <input type="text" id="fullname" name="fullname">
</div>
<div class="form-item column2">
  <label for="useremail">Email:</label>
  <input type="email" id="useremail" name="useremail">
</div>
```

4. Give the select (browser options question) and checkbox (pet question) **div** containers an additional class name of **column1**.

Step 4: CSS Style the Class Selectors

1. Style the **.form-item** class selector to have a **display** of **grid** to align the label and input/select elements. Also apply **align-items: center** for vertical alignment in the grid layout.

▼ Example

```
.form-item {
  display: grid;
  align-items: center;
}
```

2. Style the **.column1** class selector to have a **grid-template-columns** of **3fr 1fr** so that the first column (the label) takes three times the space of the second column (the input). Add a margin as needed to achieve the desired spacing.

▼ Example

```
.column1 {  
  grid-template-columns: 3fr 1fr;  
  margin: 0.5rem 0;  
}
```

3. Style the `.column2` class selector to have a `grid-template-columns` of `1fr 2fr` so that the first column (the label) takes half the space as the second column. Also, add a margin as needed. You can adjust this value to get the desired spacing.

▼ Example

```
.column2 {  
  grid-template-columns: 1fr 2fr;  
}
```

Step 5: Test and Adjust

1. Be sure to review your work in the browser using **Five Server**.
2. Add additional CSS rules and declarations as needed to get consistent spacing and uniform alignment.

Submission

1. Commit and push your work to your wdd130 GitHub Pages enabled repository.

Week 05